Data Analytics Environment: Combining Visual Programming and MLOps for AI workflow creation

André Grilo
UNINOVA—Centre of Technology and Systems (CTS)
Caparica, Portugal
a.grilo@uninova.pt

Paulo Figueiras
UNINOVA—Centre of Technology and Systems (CTS)
Caparica, Portugal
paf@uninova.pt

Bruno Rêga
UNINOVA—Centre of Technology and Systems (CTS)
Caparica, Portugal
b.rega@uninova.pt

Luis Lourenço
UNINOVA—Centre of Technology and Systems (CTS)
Caparica, Portugal
lcl@uninova.pt

Amin Khodamoradi
UNINOVA—Centre of Technology and Systems (CTS)
Caparica, Portugal
a.khodamoradi@uninova.pt

Ruben Costa
UNINOVA—Centre of Technology and Systems (CTS)
Caparica, Portugal
rddc@uninova.pt

Ricardo Jardim-Gonçalves
UNINOVA—Centre of Technology and Systems (CTS)
Caparica, Portugal
rg@uninova.pt

2024 IEEE International Conference on Engineering, Technology, and Innovation (ICE/ITMC) | 979-8-3503-6243-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICE/ITMC61926.2024.10794244

***Abstract*—In the Industry 4.0 scene, Artificial Intelligence (AI) is sought after as a new way of getting a competitive advantage from other market competitors. This technology can support not only in-line production status assessment processes, which enable a better control over the quality of the final product, but also to identify potential bottlenecks and other inefficiencies that can exist or occur in production processes. However, this technology has some obstacles that make its access difficult for businesses that do not have the necessary resources for implementing AI solutions, whether due to the intrinsic difficulty to handle such technologies, which require specialists (engineers, data scientists) that are not normally part of industrial human resources, or due to the integration and management of these technologies with already established processes and environments. To approach these technological accessibility challenges, some concepts are being applied, such as in the case of no code/low code solutions, i.e., the reduction or complete removal of programming requirements while using these technologies, and Machine Learning Operations (MLOps), where the integration and lifecycle management of these solutions use the same approach as DevOps but applied and adapted to AI technologies. This paper presents an innovative, open-source and scalable approach towards AI pipeline creation, integration, and lifecycle management in Industry 4.0 scenarios, in which these no code/low code and MLOps concepts are used, as well as a real-life application in the manufacturing industry.**

Keywords—MLOps, AI workflows, No-code/Low-code platforms, Industry 4.0

I. Introduction

Industry 4.0 is introducing many disruptions in the manufacturing landscape, not only by changing the way methods and processes are implemented in a factory, but also through the integration of new technologies into production processes [1]. One of the more prominent technologies that enterprises want to add to their catalogue of available tools, is Artificial Intelligence (AI). The reason why AI is sought out as an essential tool is due to the fact that factories and production processes are becoming increasingly more data driven, as many other technological paradigms that are being introduced into the Industry 4.0, such as Big Data, Digital Twins and Cyber Physical Systems, increase the heterogeneity and quantity of data that is available to manufacturing stakeholders [2] [3].

Access to data from all production steps, not only about products but also the status of the infrastructure as well, opens new possibilities regarding management of the factory as a whole, through data analysis, where it is possible to increase production efficiency, support decision-making and perform real time predictive analyses [4]. This can be achieved by feeding various types of data, through data pipelines, to AI analytical tools, which will extract and provide insights over processes, products and other business indicators that are being monitored [5]. The two main outcomes expected from applying AI in the production process are optimisation of production, which in turn corresponds to an increase in revenue, and the reduction of waste and energy spending, lowering production costs.

Even though AI can provide many advantages to industrial manufacturers and businesses, the technology is not easily accessible for the most part, even with the increase in its demand. The main challenges regarding the usage of AI are mainly connected with the complexity that comes coupled with it. From the development of AI tools, their integration with the systems and pipelines that are already in place, their maintenance and life cycle management, and the interpretation of the raw results that AI analytical algorithms provide, all of these can be seen as barriers that limit the access of companies to AI tools, depending on their available resources. This is due to the fact that AI requires expertise in many areas, like data engineering and machine learning, which are not part of the catalogue of skills associated with factory personnel, and the size of the company or enterprise is directly connected to the difficulties of accessing these technologies and the appropriate human resources to apply them. For this reason, small and medium enterprises (SMEs) are at a clear disadvantage in terms of competition over the usage of these technologies, when compared to big enterprises, reducing their capacity of surviving in their market.

Both in the scientific literature and in the market of AI platforms, the main approaches that are being used to facilitate the usage and integration of AI technologies are the development of no-code or low-code platforms [6], and also

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

the application of the Development Operations (DevOps) methodology applied to AI technologies, also called Machine Learning Operations (MLOps) [7]. The low-code/no-code approach enables the abstraction of some of the inherent complexity of these technologies, such as reducing or removing the code-writing knowledge requisite, while the MLOps methodology enables the development, integration, and lifecycle management of these platforms as increasingly automated processes.

This work proposes a solution to tackle the aforementioned challenges by providing a framework that strives to make AI more accessible to non-experts through the application of MLOps and a no-code/low-code approach. By using these two techniques it is possible to introduce an abstraction layer over two focal points related to AI pipeline creation. On one hand, a low-code/ no-code approach ensures that all the programming code associated with data-driven tasks, such as data processing, data cleaning and analysis, is pre-built or pre-written into blocks, which in turn abstracts complexities related to traditional coding and software development. Adding these blocks to a User Interface (UI) allows for a dynamic and intuitive way of creating AI pipelines through block-based visual programming, using drag and drop functionalities, which is a particular example of low-code/no-code approaches. By simply dragging and dropping these pre-built blocks within a UI, individuals with varying levels of technical expertise can assemble AI pipelines without the need for extensive software development knowledge. On the other hand, MLOps will introduce automation and simplification over AI pipeline management tasks, such as handling orchestration or scheduling, without requiring constant user interaction, and lifecycle management, providing a platform that keeps track of the maintenance of the created AI pipelines. Besides pipelines, MLOps approaches also enable the management of AI or Machine Learning (ML) models, where the same aspects of lifecycle management are applied, but over the context of AI/ML models, like keeping track of multiple versions of the same model over different training iterations, as well as the associated model precision scores, enabling a constant evaluation and monitoring of the models.

A detailed description of each of the previously introduced components will be given in section III (Research Approach), elucidating the purpose of each component and how it removes layers of complexity brought by regular AI development processes. But before presenting the overall solution, section II (State of the Art) will present a literature review over AI pipeline definition approaches, regarding their creation, orchestration, lifecycle management and execution, as well as methodologies and technologies that have emerged as main driving forces in the development and operationalization of AI-driven data pipelines. A benchmark of the current low-code/no-code solutions for AI pipeline creation present in the market is also presented in the State of the Art section. The rest of the document is organised as follows: section IV (Findings) contains two example use cases in which the proposed framework was deployed to perform AI-based analyses over data from two factories, providing validation means in terms of its application in real-world industrial environments; and section V (Conclusion) discusses the findings and conclusions as well as future work prospects for the proposed solution.

II. State of the Art

With the growing interest around areas such as AI and ML [8] and with the surge in development of AI-based tools also comes an increase of available tools that aim to support and ease the creation of AI and data-driven pipelines, or workflows, for non-experts. This new set of solutions aim to ease the processes of creating AI or data processing pipelines and streamlining the orchestration, execution and serving of such pipelines, i.e., handling the execution, scheduling, management, and monitoring of AI workflows [9]. Some of these solutions support non-experts through a no-code/low-code philosophy, enabling the creation of data processing workflows using advanced AI/ML technologies, alongside data cleaning processes [10], while also reducing or removing entirely the necessity of writing software code. These no-code/low-code approaches are realized by allowing the creation of AI workflows through a UI via drag and drop functionalities of visual components or blocks that represent actual data processing tasks. The blocks that are made available in the UI represent tasks that are associated with data processing or AI, such as data cleaning or AI/ML model training and come prebuilt and prepacked so that the need of writing technical code for the required tasks is removed.

This practice of adding and linking elements in a UI is also referred to as “visual programming”. In this scenario, the visual programming of an AI workflow is done by putting together multiple blocks pertaining to different data processing tasks, whether they are tasks for data collection, data transformation, training and serving AI models, creating a flow of data throughout the pipeline [11]. By following a visual programming methodology and using modular blocks that can be executed independently, these platforms play a pivotal role in data analytics and are transforming the way tools for AI-driven analytics are being developed while, with the increasing demand from enterprises and industries from enterprises and industries for simplistic tools that are powered by AI/ML technologies, software developers are prone to developing these types of tools.

The current methodology revolves around AI model-driven engineering, where the usage of blocks alongside visual programming environments removes the complexity of designing AI models, by making the environment intuitive and easy to use. This approach makes the development process simpler and increases accessibility to these demand-growing technologies [12]. And a consequence of this is the boom in the number of data-driven and AI-driven workflow creation environments.

AI pipelines represent many steps from data analytics processes, where data flows from data collection and preprocessing, to feature extraction and model selection, and finally to model training, evaluation, deployment, and monitoring. As mentioned in [13], these steps are laborious, time consuming and require specific data science and programming skills, whether they are related to the development of the required tasks, mainly through programming, choosing the most adequate model for the scenario at hand, or lifecycle management of the pipeline, such as its orchestration or scheduling. To reduce the complexity of AI pipeline creation, and to add or increase automation of the orchestration, deployment, and management, two methodologies are often introduced in the literature: MLOps and no-code/low-code platforms. The next

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

two subsections review these concepts, and how can they decrease or remove the complexities mentioned previously.

1) MLOps

MLOps is a methodology that is becoming more prominent in the AI scene. Even though it is a recent concept, MLOps draws heavily from the principles and practices established in DevOps [14]. DevOps, short for Development Operations, is a methodology used in software development and in the Information Technology (IT) industry, where the paradigm revolves around the union between the development stage and the operations stage, through the application of tools and common practices, aiming to reduce the time associated with development and deployment, while also promoting flexibility and automation between each stage. Bringing these DevOps concepts into the AI and machine learning fields supports the increase of efficiency and effectiveness of machine learning development, deployment, and management. This combination between DevOps and AI/ML is called MLOps.

MLOps is not only a driving factor in the creation of a seamless collaboration between data scientists and operations teams, enabling faster iteration cycles, improvement in performance and greater scalability of AI-driven frameworks, but can also remove some of the complexity associated with lifecycle management of AI/ML solutions [13]. The authors of [15] present a study over the MLOps methodology based on the multiple fields that it covers, alongside some technologies and frameworks that exist and are based in this concept. These technologies, besides the division between open-source and commercial, can be divided regarding the functionalities that they offer in the context of MLOps services, from providing a comprehensive framework for bundling multiple ML technologies and libraries to deliver their functionalities, such as IBM Cloud Pak [16] for data, to offering a platform for orchestrating and managing the lifecycle of AI pipelines, ensuring efficient development, deployment, and monitoring, such as Apache Airflow [17] or Amazon SageMaker [18], going through platforms that focus on the lifecycle management of AI/ML models, enabling functionalities such as model saving, serving, and monitoring to ensure optimal performance and scalability, such as MLflow [19].

In the context of AI pipeline creation, all these capabilities provided by the MLOps methodology enable the simplification of handling AI/ML technologies, through automation of many of the steps connected to development, deployment, orchestration, and monitoring. The authors of [20] developed a tool that uses this methodology to automate the process of deployment of both solutions, where all the configuration, installation and initialization steps are abstracted from the user, and also AI pipelines, handling their orchestration and execution. The tool also provides an UI where it is possible to define an analytical pipeline, albeit this definition is done in text format, where the user is required to edit a JSON file with the desired parameters. Even though the usage of MLOps is evident here, with many of the steps connect with deployment, lifecycle management and orchestration automated, facilitating the usage of AI pipelines, this solution does not provide a dynamic and intuitive way for the definition of the aforementioned pipelines.

Another similar solution in which MLOps is used as a means to facilitate the deployment and orchestration of AI pipelines is presented in [21]. Here, the approach revolves around using a MLOps pipeline to execute analytical processes in a more flexible and seamless manner, as well as its deployment and lifecycle management in a production environment. The tool leverages two MLOps technologies that were already mentioned above, Apache Airflow and MLflow. Airflow is used as the orchestration and lifecycle management framework for AI pipelines, overseeing the aspects of execution, scheduling, and monitoring. These AI pipelines are represented in Airflow as workflows. In contrast, MLflow serves as the designated framework employed for the registration, storage, versioning, and deployment of AI/ML models, since MLflow offers a comprehensive suite of tools for managing the end-to-end machine learning lifecycle, including experiment tracking, model packaging, and model serving. As with the previous solution, even though this tool leverages the power of MLOps to simplify and automate the process of deployment, orchestration, and management of AI pipelines, it still presents limitations regarding the AI pipeline in two ways. The first one is that to bridge the connection between MLOps technologies, this tool uses an API without any UI, increasing difficulty to non-expert users. The second limitation relates to Airflow, in which the definition of AI pipelines as workflows is done programmatically, i.e., by writing software code. This is also another barrier regarding non-expert users' use of this solution for their AI pipeline creation needs.

Even though MLOps is a big contributor to increase the accessibility of AI, and in this case, analytics through AI pipelines, there are still some limitations related to the AI pipeline definition that can introduce obstacles to non-expert users. As such, the next subsection will present a concept that, when used alongside MLOps, could be used to decrease the aspect of pipeline definition, increasing the overall accessibility of analytics and AI through AI pipelines: no-code/low-code platforms.

2) No-code/low-code Platforms

As presented in the previous subsection, one of MLOps main flaws is linked to AI pipeline definition. The creation of pipelines to perform some analytic steps requires technical code to be written, as in the case of defining AI pipelines as workflows in Airflow, in some cases, through text in files such as JSON, where technical knowledge is required. To address this limitation, no-code/low-code platforms are used, significantly reducing the technical requirements to design data-driven pipelines.

A no-code/low-code platform is a software development platform that enables the creation of applications and workflows with minimal or no software code writing, normally using a visual UI. This solution adds a layer of abstraction over the programming languages, creating a higher-level form of programming and reducing the requirements for technical knowledge, while also increasing the development speed [22]. Depending on the requirements, possible approaches for the development of these platforms can vary, since there are many characteristics that can define no-code/low-code approaches, and different perspectives need to be taken into account [23]. One approach that allows the creation of no-code/low-code platforms is the concept of block-based programming.

Block-based programming a visual programming paradigm in which the development of programs is achieved by linking blocks together, with each block symbolising a visual representation of already pre-written, pre-packaged

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

functional software code. A block-based programming environment provides a visual interface where the blocks can be dragged-and-dropped onto a workspace, and when these blocks are interconnected, they can be compiled into a sequenced software program. The features from no-code/low-code that are associated with visual programming approaches based on blocks are the introduction of abstraction, the drag-and-drop functionalities and usage of visual tools, and the main users for these platforms are non-programmers, as presented in [22].

This methodology is already used as a means of introducing programming to students. By having blocks instead of lines of code, it facilitates aspects related to programming, such as making it easier to read and comprehend, visually representing program functions, and easing the process of building software code using these blocks [24]. The same concept of block-based programming can be applied to data analytics and AI pipeline creation, in which "chunks" of AI and data-driven code can be encapsulated into blocks that represent data-driven tasks such as data analysis, data connection, data cleaning or loading data into a model, and the user only needs to drag and drop these blocks and link them together to create AI and data-driven pipelines, via a UI [25]. The application of this paradigm when developing platforms for the creation of AI pipelines can significantly reduce the complexity associated with this process [25], and by complementing this methodology with MLOps opens up possibilities of increasing the accessibility to AI technologies, while also leveraging all of their potential [26].

The authors in [27] present a solution that uses both concepts of MLOps and no-code/low-code for the creation of AI pipelines to simplify and automate many of the steps regarding AI pipeline definition, deployment, scheduling, and monitoring. Another example is presented in [28], in which a more complete solution is presented in regards to the application of block-based programming for the creation of AI pipelines, by using a workspace to define the pipeline, and the platform provides functional blocks that represent different analytical tasks that can be dragged and dropped into the workspace. The execution of the pipeline is also automated, since the user is only required to, after choosing the blocks and interconnecting them, click on a button to run that same pipeline. All these functionalities are provided in the UI, and the requirements for coding are all removed.

Besides solutions in the literature, there are also commercial platforms that provide functionalities for AI pipeline creation applying the concepts of low-code/no-code and MLOps. Some of these platforms are introduced in Table I, which presents a benchmark on some aspects that are relevant for comparing such platforms. The chosen tools for this benchmark were Azure ML [29], Microsoft's cloud-based ML platform, RapidMiner [30], a data science platform for data science and analytics, Orange [31], an open -source tool for data visualisation, exploration and analysis, Knime [32], another open-source solution that provides a data analytics, reporting and integration platform, Dataiku [33], a centralised platform for the development, deployment and management of data science projects, and Google Cloud AutoML [34], a suite that provides ML products and functionalities. The selection of these tools for the benchmarking analysis is based on several factors, such as the time in the market, where tools like Orange and RapidMiner are present for decades versus Google Cloud AutoML which is new, their market presence and the size of the developer or owner, and if they are a cloud service, such as AzureML, or if they can be deployed whether on premise or on cloud, such as Dataiku or Knime.

TABLE I. NO CODE LOW CODE AI PIPELINE CREATION FRAMEWORKS

AI Tool	Scalable	Open source	Built with open-source software	Customizable
AzureML	Yes	No	Partial	Limited
RapidMiner	Yes	Yes[a]	Yes	Yes
Orange	Limited	Yes	Yes	Yes
Knime	Yes	Yes[a]	Yes	Yes
Dataiku	Yes	No	No	Yes
Google Cloud AutoML	Yes	No	No	Yes

[a.] The free version has limited functionalities vs paid version.

III. RESEARCH APPROACH

The research presented in this paper addresses the challenges presented in the previous sections regarding the complexity of adoption of AI technologies without the involvement of AI experts, such as data scientists or engineers. By developing a framework that integrates MLOps practices alongside low-code/no-code development approaches in the creation of an AI workflow creation platform, and by evaluating its application in real-world scenarios to validate the streamlining of deployment processes, the overall accessibility towards AI technologies is increased, leading to a bigger adoption and utilization of such technologies. This, in turn, enables the possibility to evaluate the impact that these practices have in the acceleration of AI democratization and the facilitation of integration of AI innovations.

The main outcome of the presented work is an innovative framework, called Data Analytics Environment (DAE), which enables the creation of AI and data-driven pipelines through a dynamic workflow creation environment. This environment provides a straightforward and intuitive approach, without requiring specific coding skills, to create workflows through a block programming interface, in which blocks are dragged, dropped and linked between them to generate end-to-end data-driven workflows. DAE utilizes powerful open-source frameworks like MLflow and Apache Airflow to effectively manage orchestration and lifecycle processes. It integrates a variety of data analytics and AI technologies, such as TensorFlow [35], Apache Spark [36], and Apache Kafka [37]. These technologies offer functionalities for data processing, analysis, and AI model creation and serving. MLflow takes care of all AI/ML model management functionalities, such as saving and serving AI models, while Apache Airflow handles the orchestration, scheduling, execution, and monitoring of data-driven workflows. Furthermore, this framework abstracts all the aspects of data analytics, integration, scalability, and lifecycle management from the user, thus removing layers of complexity from these characteristics.

The conceptual architecture of DAE is depicted in Fig.1.

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

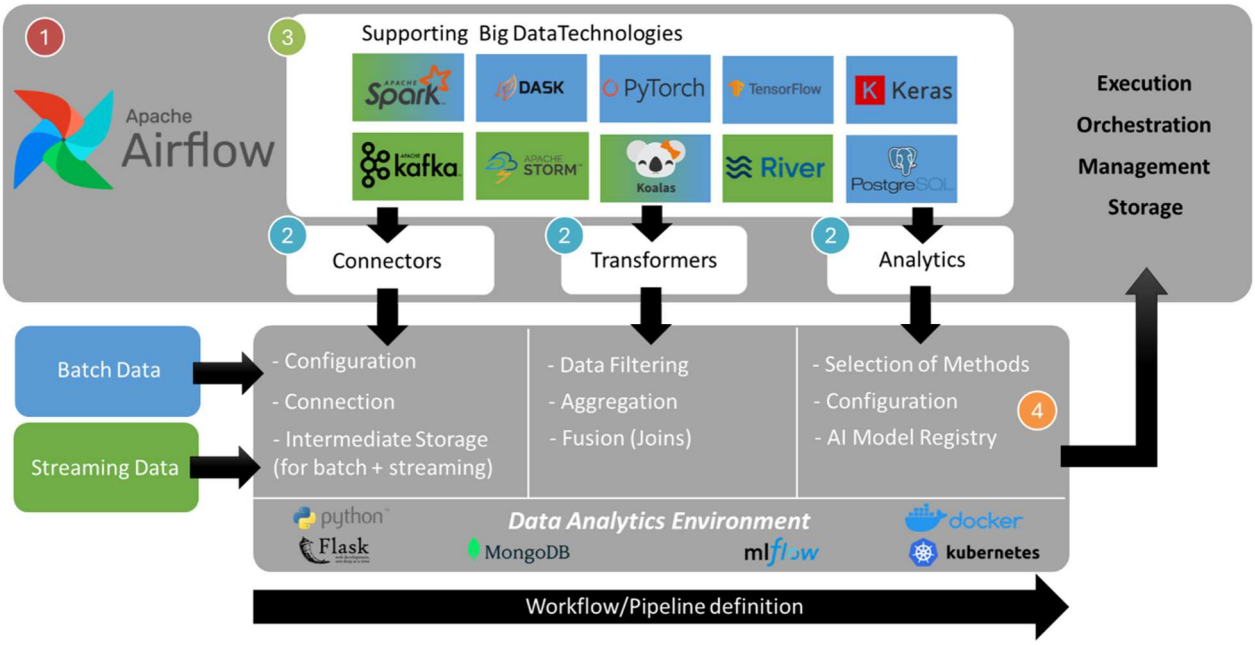

Fig. 1. Data Analytics Environment framework architecture

A. Architecture

The DAE framework (Fig. 1) comprises four main elements: the Apache Airflow platform (1), a set of block operators (2), a set of supporting technologies (3), and the User Interface (4). The first element, the Apache Airflow platform, serves as the orchestrator for AI workflows. It is responsible for scheduling, executing, and monitoring tasks and workflows, it supports integration with various open-source, scalable AI and Big Data technologies, such as Apache Spark, Dask [38] or Koalas [39]. Airflow conceptualizes workflows as Directed Acyclic Graphs (DAGs), a type of a data structure composed of nodes and edges, in which each node represents a discrete task or process, which are atomic in nature, and are interconnected by edges, creating dependencies between nodes, enabling data flow throughout the system. One of the main properties that enables Apache Airflow DAGs to be the representation of workflows, is the fact that they contain no directed cycles, meaning there is no path that starts and ends at the same node. This ensures that tasks in the graph can be executed in a specific, sequential order without encountering circular dependencies.

Nevertheless, Airflow presents some limitations in terms of its ease of use. The first limitation is connected to how Airflow handles the information flow in a DAG. Each node is atomic and independent from the other nodes, and all the data that is inside that node, whether it was loaded, transformed, or applied to a ML model, cannot be directly shared, or sent to the next nodes. This limitation hinders one of the focal points of AI pipelines, which is the fact that there is no dataflow in Airflow DAGs. To tackle this limitation an intermediary storage database is used, acting as a supporting technology, so that for each step of the workflow, the data can be loaded and saved between nodes. The intermediary storage chosen for this end was MongoDB [40], a NoSQL, document-oriented database. MongoDB was chosen for its schemeless architecture, which makes it independent of the data type or data source used in the workflow, thereby offering flexibility and adaptability to varying data structures and sources.

The second limitation of Apache Airflow lies in the need for programmatically define these nodes and edges in a DAG. To address this limitation, an abstraction layer was created to remove the requirement of writing technical code to define DAGs. This is achieved by using the DAG Factory [41], a Python library that enables the dynamic generation of DAGs from YAML configuration files, allowing the structure and dependencies of the DAG to be specified in a declarative manner, where nodes are represented as YAML objects containing properties that enable the "translation" of the file into an Airflow DAG. The usage of the DAG Factory also enables the encapsulation of software code into blocks, with each block being equivalent to an atomic task from a DAG, and these blocks are associated with AI or data workflow tasks. These blocks of code are categorized as operators, which correspond to the second element of the framework.

Operators are divided into three categories, delineated in Fig. 1 within the Airflow component, each representing the main processes of data analysis: connection to data, data transformation and application of analytics, AI, and ML. These categories are connectors, transformers, and analytics operators. Connector operators handle data collection from different data sources, such as files or databases, making it available to the next steps of the workflow, through the selection of the suitable connector and the inputted data source configuration by the user. Besides being able to handle data collection from files or databases, as mentioned above, which represent batch data, these operators can also handle data collection from streaming data sources, such as Apache Kafka. Transformer operators contain all procedures connected to data transformation and data preparation, ensuring that the relevant data is in a usable format for further processing and application in subsequent AI and ML tasks. Examples of transformers include cleaning, filtering, aggregation, joining and normalization tasks, just to name a few. The last category is the analytics operators, encompassing various Machine Learning algorithms, providing functionalities both for training ML models and other data analytics tasks. These operators utilize MLflow to support these tasks, providing enhanced capabilities for model

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

life cycle management, such as saving, serving, versioning and monitoring.

Every category of operators comes prepacked with relevant technologies that support them, as seen in Fig. 1, such as, for instance, Apache Spark and Apache Kafka for the connectors, enabling batch and streaming data connection, Koalas and Dask for the transformers, allowing distributed computation to handle big amounts of data, and TensorFlow, Spark MLlib and PyTorch [42] for analytics, providing a wide range of the most common ML algorithms from each framework. The set of supporting technologies allows the integration of all these technologies and frameworks into the DAE framework, leveraging their functionalities from all relevant areas that relate to the fields of AI and data analytics. The solution provides not only all the instances for these technologies internally, creating a seamless experience while interacting with them, but it also provides a technology instance configuration feature, where it is possible to connect to external instances, allowing the utilization of existing infrastructures and services, enhancing the flexibility and customizability of the platform.

The final element of the DAE framework is the workflow creation environment, i.e., the UI that allows users to define their AI and data-driven pipelines into executable workflows, through visual programming via drag-and-drop of the operator blocks, provided in the aforementioned categories, into a workspace, where all the configuration of these operators is done. The UI provides support for the creation of new workflows, saving workflows, loading workflows, and initializing workflows to be executed in the Airflow orchestrator. As previously mentioned, the framework utilizes the DAG Factory to interpret YAML configuration files into Apache Airflow readable workflows, and the user interface handles the creation of these YAML files that define all the workflow aspects (type of operators, configuration parameters, dependencies), creating a seamless user experience between the workflow creator environment UI and the orchestrator software, Apache Airflow. The dynamic interaction between these two elements also ensures that any change made in the workflow through the UI, when it is reinitialized, will appear in the Airflow workflow, without needing to redo the whole workflow.

The configuration of the external technologies is also done through the framework's UI, by providing the input parameters for connecting to the external technology instances. The DAE framework is also customizable regarding operators, allowing the user to add their own custom operators to the environment, by providing the necessary resources to the corresponding interface. The resources to add a new operator are the name of the operator, a Python script file that contains all the necessary software code to perform the task, and a JSON file with the metadata for that operator. The metadata defines various aspects of operators, such as their name, and description, as well as the type of operation and the technology powering the operator (e.g., Spark). Additionally, it specifies the parameters required for the operator's execution, along with the corresponding presentation names and descriptions for each parameter.

All of the services and components that support the DAE framework were developed in Python and are deployed as Docker containers [43] or Kubernetes pods [44], including the supporting technologies such as Spark, making the tool easy to install in any environment, while also being easy to customize by directly editing the docker file of each component.

B. *Benchmark with existing frameworks*

The characteristics of the DAE framework and a comparison to similar frameworks can be seen in Table II, considering the benchmark presented in Section II.

TABLE II. BENCHMARK WITH PROPOSED SOLUTION

AI Tool	Scalable	Open source	Built with open-source software	Customizable
AzureML	Yes	No	Partial	Limited
RapidMiner	Yes	Yes[a]	Yes	Yes
Orange	Limited	Yes	Yes	Yes
Knime	Yes	Yes[a]	Yes	Yes
Dataiku	Yes	No	No	Yes
Google Cloud AutoML	Yes	No	No	Yes
DAE	Yes	Yes	Yes	Yes

The DAE framework is open-source, customizable, scalable, and can be integrated with several existing technologies and frameworks. Furthermore, all of the components are built using open-source technologies, such as Apache Airflow, for the orchestration of workflows, or MLflow, used for life cycle management of AI models. The DAE framework is ready to be deployed as a standalone tool, since it comes bundled with the necessary technologies and other tools to build and execute the most common AI workflows and algorithms, but it also enables integration of custom code blocks and it enables the connection to externally deployed technology instances, such as databases or other AI/ML technologies, such as TensorFlow or Apache Spark. A final characteristic of this tool is the fact that all its components are containerized, through the usage of Docker or Kubernetes, meaning that deploying the DAE framework can be done on different environments (Cloud or on-premises servers), making the framework multitenant.

IV. FINDINGS

The validation of the DAE framework was done in the context of i4Q, a European Commission-funded research project connected to Industrial Data Services for Quality Control in Smart Manufacturing. The i4Q project had a use case in Riastone, a tableware ceramics factory located in Portugal. Riastone is a factory from the "Visabeira Group" that produces tableware, mainly plates, and bowls, and is one of the main providers of these products to IKEA in the European market. They produce annually 50 million pieces of tableware, where 99% of these are for exportation. Their use case is related with their Overall Production Effectiveness (OPE), an indicator that measures the number of products that can be sold versus the number of products produced. Their current OPE is ~92%, which is turn means that from the 50 million pieces produced in a year, 4 million are defects that cannot be sold, meaning that some of the resources used are not getting the expected return of the investment made.

Riastone personnel, with a strong expertise in the production process but limited data analytics knowledge, used the DAE framework to create their own AI workflows. These

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

pipelines established a connection between the quality of the final product and different stages of the production process, in which the knowledge over both the data and production processes was instrumental. Some support regarding the usage of the framework was provided by the authors of the paper.

Two workflows were defined, each related to an indicator that was identified by the factory personnel as possible factors that affect their OPE. A complete description of the indicators, as well as the details of the defined pipelines will be presented in the following two subsections.

A. Raw matter composition

One of the identified indicators was the composition of raw matter. When this composition deviated from the norm due to the raw matter not being homogeneous or varying humidity levels within the storage silo, for instance, it could result in defects in the final product. Such defects ranged from cracks and fissures to deformations, including warped tableware. The raw matter's composition was measured by performing a spectrometry analysis on the ceramic paste that is used to produce the tableware products. The resulting data from the spectrometry analysis was then used to train a model that correlates final product quality, and OPE, with the raw matter's composition.

For the analysis and prediction of the product quality based on the raw matter, it was necessary to identify what are the data sources, what data was being used, what preprocessing was necessary, and the type of AI/ML model that was going to be used. After having all the above information, the next step was the definition of the workflows, one for the training of the ML model and the other to serve the created model to perform prediction. The data sources required for this workflow were the data from the raw matter's composition, which was provided through the spectrometer analysis, and the respective quality data.

The raw matter's composition data was provided in Excel files and comprised the spectrometry values, i.e., the light absorbance values at discrete wavelengths for the raw matter that is used in the production process. In this way, different absorbance values at different wavelengths provide information about the composition of the raw matter, as well as data related to the humidity, or water content of the matter. The quality data was provided in CSV format and comprised of data regarding the type of defects.

For the data transformation step, a custom Python script was written and added as a new transformer block, in order to handle the specific formatting of the Excel files, and a generic join to merge the two datasets from the two files. The analytical model chosen to be trained was a regression model from the LightGBM framework [45]. To add this new operator, the "Add Custom Operator" functionality was used, which enables the addition of new custom operators by providing a code script, a metadata file, and a unique name for the operator.

All the operators, except the aforementioned transformer operator, were available in the DAE workflow creation workspace. The connection to the data was done using the "File Connector" operator, which supports CSV and Excel files. The next operator used was the "Concat Dataframes" operator, a prebuilt operator that allows the joining of two datasets into a unique data frame. The last step after merging the two datasets into one was the training of the model, using the "LGBMRegresssor Train" operator. This last operator not only trained the model, but also managed the storing process of the model into MLflow.

To use this model, another workflow was necessary, with the same operators except the last one, where instead of the "LGBMRegresssor Train" operator", the "LGBMRegressor Predictor" operator is used, as depicted in Fig. 2. The main difference between these two operators is that the first one, as was mentioned, trains and stores the model into MLflow, while the second fetches the previously trained model, also from MLflow, in order to be used. The generated data from the prediction was then stored into a database, using the saving capabilities of the "RDBMS database" operator, a type of connector for relational databases.

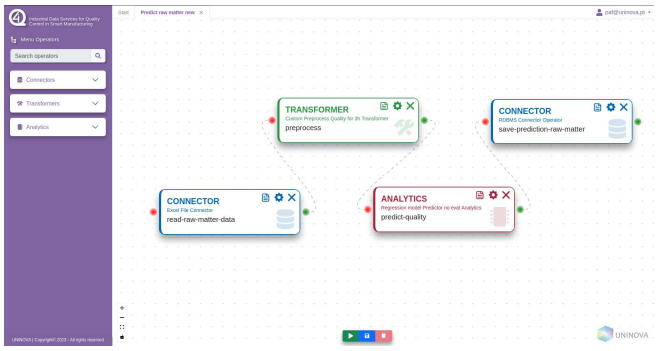

Fig. 2. Raw matter quality prediction workflow in DAE

B. Glazing process

The other identified indicator pertained to the glazing process, which involves spraying paint onto a plate to provide colour and coating. The plate is rotated on a platform while a spray gun applies the paint. To ensure proper operation, the glazing liquid must maintain specific levels regarding some of the liquid's parameters, such as viscosity, temperature, and density. Deviations from the acceptable ranges for these parameters can lead to defects such as poor finishing, incorrect colour tones, or dripping, which negatively impact OPE.

The glazing liquid's properties were measured with sensors in the production line, which were stored in the factory's dedicated database. This data was then used to perform a correlation analysis with the product quality data and OPE.

The analysis and prediction of the product quality based on the glazing step followed the same procedure as the one based on raw matter composition.

The data corresponding to this stage of the production process contained information regarding the viscosity and the density of the glazing paint inside the reservoir, the density and the temperature of the glazing paint when is being ejected, and the turning speed of the tableware being painted. This data was provided in CSV format.

For this case, the workflow to train the model for the prediction of the quality required more operators to be used, and the final workflow comprised a total of 8 operators, both prebuilt and custom. The operations required for this analysis revolved around removing extra columns of information from the CSV file that were not used, as well as multiple steps of encoding. The encoding steps were used to transform categorical data, such as the list of defects, into numerical

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

format using techniques such as one-hot encoding, making the data suitable for input into the machine learning model.

The chosen model for this analysis was a Decision Tree model, which already exists as an available operator in DAE.

Two workflows were also created for the glazing liquid properties-based predictive analysis: the first one was to train the model, through the usage of the “Decision Tree Train” operator, and the second one to perform the prediction, by using the “Decision Tree Predictor” operator. As with the previous workflow for the raw matter, both the “Decision Tree Train” operator and the “Decision Tree Predictor” operator interacted with MLflow for the storing and serving of the trained model, respectively. The generated data from the prediction was also stored into the same database, using the “RDBMS database” operator write functionality. In Fig. 3 is depicted a simplified version of the prediction workflow, as the preprocessing was done. In Fig. 3, a simplified version of the workflow is depicted, as most preprocessing tasks were completed beforehand, stored in the database, and then loaded using the read capabilities of the "RDBMS Connector" operator.

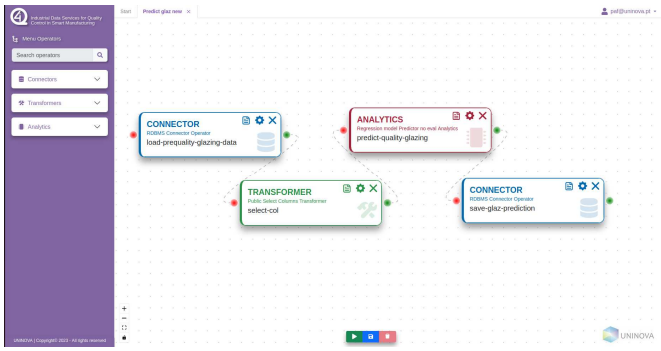

Fig. 3. Glazing quality prediction workflow in DAE.

C. Match and contribution

This paper contributes to the IEEE Technology and Engineering Management Society (TEMS) by introducing a framework that integrates MLOps with no-code/low-code platforms to democratize AI technology, making it accessible and manageable for non-experts. This dual-layered approach simplifies AI pipeline creation and lifecycle management through intuitive drag-and-drop interfaces, aligning with TEMS's goals of advancing technology management and innovation. Validated in industrial settings, our framework demonstrates tangible improvements in productivity and quality, providing practical evidence and comprehensive analysis that supports TEMS members in navigating and implementing AI technologies effectively.

V. Conclusion

The development of low-code/no-code AI platforms for AI pipeline creation is becoming more prominent due to the high demand from the industrial sector for tools that require less specific knowledge to access AI technologies. The DAE framework exemplifies this trend by integrating MLOps with a no-code approach, enabling users to design and create AI workflows through a user-friendly interface. While this framework has made significant strides in accessibility, there are still limitations to be addressed. This conclusion section outlines the current limitations, summarizes the contributions of the DAE framework, and discusses potential future work to enhance its functionality and user experience.

A. Limitations

In the current iteration of the DAE framework, while it effectively removes the need for programming skills, it still requires users to have a basic understanding of data science to define the necessary steps for specific data analysis workflows. Users must also be knowledgeable about the most appropriate machine learning algorithms for their tasks and handle model parameterization, despite most models coming with default parameters. Another limitation is that after building and setting up a workflow, users must navigate to the Apache Airflow UI to execute it, which can be cumbersome.

B. Concluding Remarks

The development of low-code/no-code AI platforms like the DAE framework is increasingly important due to the high demand in the industrial sector for tools that simplify access to AI technologies. The DAE framework combines MLOps with a no-code approach, providing a UI-based workspace and prebuilt operator blocks for data processing and analysis tasks. By integrating tools such as Apache Airflow for workflow operations and MLFlow for managing the lifecycle and versioning of AI/ML models, the DAE framework represents a significant step towards making AI more accessible to users with limited programming skills.

C. Future Work

Future development iterations of the DAE framework will focus on three main aspects: first, the creation of a recommender system (e.g., a chatbot) to assist users throughout the workflow definition process by providing guidance on necessary steps, data transformations, suitable data analytics methods, and optimal model parameterization; second, the full integration of Apache Airflow and MLFlow into the DAE platform, enabling users to access all functionalities within a single UI without switching between applications; and finally, the validation of workflows and operator parameters to ensure error-free execution and improve the overall user experience.

Acknowledgment

The authors acknowledge the contribution of the European Commission-funded Horizon 2020 research project i4Q (Grant agreement ID: 958205) and Horizon Europe research project RE4DY (Grant agreement ID: 101058384) for the development and validation of the presented work.

References

[1] L. S. Dalenogare, G. B. Benitez, N. F. Ayala and A. G. Frank, “The expected contribution of Industry 4.0 technologies for industrial performance,” *International Journal of Production Economics,* vol. 204, pp. 383-394, 2018.

[2] Z. Jan, F. Ahamed, W. Mayer, N. Patel, G. Grossman, M. Stumptner and A. Kuusk, “Artificial intelligence for industry 4.0: Systematic review of applications, challenges, and o pportunities,” *Expert Systems with Applications,* vol. 216, p. 119456, 2023.

[3] R. S. Peres, X. Jia, J. Lee, K. Sun, A. W. Colombo and J. Barata, “Industrial Artificial Intelligence in Industry 4.0 - Systematic Review, Challenges and Outlook,” *IEEE Access,* vol. 8, pp. 220121-220139, 2020.

[4] L. Duan and L. Da Xu, “Data Analytics in Industry 4.0: A Survey,” *Information Systems Frontiers,* 2021.

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.

[5] A. Raj, J. Bosch, H. H. Olsson and T. J. Wang, "Modelling Data Pipelines," in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2020.

[6] Z. Yan, "The Impacts of Low/No-Code Development on Digital Transformation and Software Development," *arXiv preprint arXiv:2112.14073,* 2021.

[7] G. Symeonidis, E. Nerantzis, A. Kazakis and G. A. Papakostas, "MLOps - Definitions, Tools and Challenges," in *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*, 2022.

[8] I. El Naqa and M. J. Murphy, What is machine learning?, Springer, 2015.

[9] K. L. Myers and P. M. Berry, "At the Boundary of Workflow and AI," in *Proc. AAAI 1999 Workshop on Agent-Based Systems in The Business Context*, 1999.

[10] J. C. Nwokeji, F. Aqlan, A. Anugu and A. Olagunju, "Big Data ETL Implementation Approaches: A Systematic Literature Review (P).," in *SEKE*, 2018.

[11] Z. Zhou, J. Jin, V. Phadnis, X. Yuan, J. Jiang, X. Qian, J. Zhou, Y. Huang, Z. Xu and Y. Zhang, *Experiencing InstructPipe: Building Multi-modal AI Pipelines via Prompting LLMs and Visual Programming,* 2024.

[12] J. M. Kanter, B. Schreck and K. Veeramachaneni, "Machine Learning 2.0: Engineering Data Driven AI Products," *arXiv preprint arXiv:1807.00401,* 2018.

[13] S. Alla and S. K. Adari, "Beginning MLOps with MLFlow: Deploy Models in AWS SageMaker, Google Cloud, and Microsoft Azure," in *What Is MLOps?*, Berkeley, CA, Apress, 2021, pp. 79-124.

[14] R. Jabbari, N. bin Ali, K. Petersen and B. Tanveer, "What is DevOps? A systematic mapping study on definitions and practices," in *Proceedings of the Scientific Workshop Proceedings of XP2016*, Edinburgh, 2017.

[15] D. Kreuzberger, N. Kühl and S. Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," *IEEE Access,* vol. 11, pp. 31866-31879, 2023.

[16] "IBM Cloud Pak," IBM, [Online]. Available: https://www.ibm.com/cloud-paks. [Accessed 15 April 2024].

[17] 'Airflow," Apache, [Online]. Available: https://airflow.apache.org/. [Accessed 5 4 2024].

[18] "Amazon SageMaker," Amazon, [Online]. Available: https://aws.amazon.com/pt/sagemaker/. [Accessed 15 April 2024].

[19] "MLflow," MLflow, [Online]. Available: https://mlflow.org/. [Accessed 5 4 2024].

[20] R. Miñón, J. Diaz-de-Arcaya, A. I. Torre-Bastida and P. Hartlieb, "Pangea: An MLOps Tool for Automatically Generating Infrastructure and Deploying Analytic Pipelines in Edge, Fog and Cloud Layers," *Sensors,* vol. 22, 2022.

[21] O. Sinkevych, Y. Boyko and L. Monastyrskyy, "MLOps based prototype of AI system for edge computing," 2022.

[22] D. Pinho, A. Aguiar and V. Amaral, "What about the usability in low-code platforms? A systematic literature review," *Journal of Computer Languages,* vol. 74, pp. 101-185, 2023.

[23] B. Alexander C and U. Frank, "Low-code platform," *Business & Information Systems Engineering,* vol. 63, pp. 733-740, 2021.

[24] D. Weintrop and U. Wilensky, "To block or not to block, that is the question: students' perceptions of blocks-based programming," in *Proceedings of the 14th International Conference on Interaction Design and Children*, 2015.

[25] N. Rao, J. Tsay, K. Kate, V. J. Hellendoorn and M. Hirzel, "AI for Low-Code for AI," *arXiv preprint arXiv:2305.20015,* 2023.

[26] L. Sundberg and J. Holmström, "Democratizing artificial intelligence: How no-code AI can leverage machine learning operations," *Business Horizons,* vol. 66, no. 6, pp. 0007-6813, 2023.

[27] Y. Wang, W. Song, Y. Yang, C. Mahmoudi, S. Shekhar and K. P. Birman, "Dash: A Low Code Development Platform for AI Applications in Industry," in *2023 IEEE 14th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, 2023.

[28] "Towards containerized, reuse-oriented AI deployment platforms for cognitive IoT applications," *Future Generation Computer Systems,* vol. 142, pp. 4-13, 2023.

[29] "Azure ML," Microsoft, [Online]. Available: https://azure.microsoft.com/en-us/products/machine-learning. [Accessed 5 4 2024].

[30] "RapidMiner," Altair, [Online]. Available: https://altair.com/altair-rapidminer. [Accessed 5 4 2024].

[31] "Knime analytics platform," Knime, [Online]. Available: https://www.knime.com/knime-analytics-platform. [Accessed 5 4 2024].

[32] "Knime analytics platform," Knime, [Online]. Available: https://www.knime.com/knime-analytics-platform. [Accessed 5 4 2024].

[33] "Dataiku," Dataiku, [Online]. Available: https://www.dataiku.com/product/. [Accessed 5 4 2024].

[34] "AutoML," Google Cloud, [Online]. Available: https://cloud.google.com/automl?hl=en_us. [Accessed 5 4 2024].

[35] "TensorFlow," [Online]. Available: https://www.tensorflow.org/. [Accessed 3 April 2024].

[36] "Spark," Apache, [Online]. Available: https://spark.apache.org/. [Accessed 5 4 2024].

[37] "Kafka," Apache, [Online]. Available: https://kafka.apache.org/. [Accessed 5 4 2024].

[38] "Lightgbm," [Online]. Available: https://lightgbm.readthedocs.io/en/latest/index.html. [Accessed 19 4 2024].

[39] "Koalas," Apache, [Online]. Available: https://koalas.readthedocs.io/en/latest/index.html. [Accessed 5 4 2024].

[40] "MongoDB," MongoDB Inc., [Online]. Available: https://www.mongodb.com/. [Accessed 5 4 2024].

[41] "Dag Factory," [Online]. Available: https://github.com/ajbosco/dag-factory. [Accessed 5 4 2024].

[42] "PyTorch," Meta AI, [Online]. Available: https://pytorch.org/. [Accessed 5 4 2024].

[43] "Docker Container," Docker Inc., [Online]. Available: https://www.docker.com/resources/what-container/. [Accessed 19 4 2024].

[44] "Kubernetes Pods," Kubernetes, [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/. [Accessed 19 4 2024].

[45] U. Gain and V. Hotti, "Low-code AutoML-augmented Data Pipeline – A Review and Experiments," *Journal of Physics: Conference Series,* vol. 1828, no. 1, 2021.

[46] "PySpark Overview," Apache Spark, [Online]. Available: https://spark.apache.org/docs/latest/api/python/index.html. [Accessed 3 April 2024].

[47] "Flask," [Online]. Available: https://flask.palletsprojects.com/en/3.0.x/. [Accessed 5 4 2024].

[48] "React," Facebook, [Online]. Available: https://react.dev/. [Accessed 5 4 2024].

[49] "LightGBM," [Online]. Available: https://lightgbm.readthedocs.io/en/latest/index.html. [Accessed 19 4 2024].

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:00:59 UTC from IEEE Xplore. Restrictions apply.